

PocketSmart AI

File 6 — Project Testing

Project: GenAI-Powered Budget & Recommendation Assistant

This document presents the testing plan for PocketSmart AI. Testing focuses on functional behavior, input validation, AI integration, user interface behavior, error handling, and end-to-end operation.

1. Testing Objectives

- Verify that the application accepts valid user requirements and budget information.
- Verify that invalid or incomplete inputs are handled safely.
- Verify that the recommendation workflow communicates correctly with the AI service.
- Verify that generated recommendations are displayed in the intended format.
- Verify that common API, network, and application errors do not crash the user interface.
- Verify that the complete user journey works from input submission to recommendation display.

2. Testing Types

Testing Type	Purpose
Functional Testing	Check whether each application feature performs its intended function.
Input Validation Testing	Check required fields, budget values, and invalid input handling.
AI Integration Testing	Check communication with the Gemini API and processing of returned responses.
UI Testing	Check controls, layouts, messages, loading states, and result presentation.
Error Handling Testing	Check behavior during API errors, unavailable services, and unexpected responses.
End-to-End Testing	Verify the complete flow from user input to final recommendation.
Usability Testing	Check whether the application flow and displayed recommendations are understandable.

3. Functional Test Cases

ID	Test Scenario	Expected Result
TC-01	Open the application.	Application loads successfully and displays the main interface.
TC-02	Submit valid category, budget, and preferences.	Request is accepted and recommendation generation starts.
TC-03	Leave a required field empty.	Validation message is shown and request is not incorrectly processed.
TC-04	Enter an invalid budget value.	Application rejects or requests correction of the invalid value.
TC-05	Submit a normal recommendation request.	AI response is returned and displayed in the result area.
TC-06	Submit a request with additional requirements.	Additional requirements are included in the recommendation request.
TC-07	Generate another recommendation.	New request is processed without breaking the application state.
TC-08	Refresh or reopen the application.	Application returns to a usable initial state according to the implemented design.

4. AI and API Test Cases

ID	Test Scenario	Expected Result
AI-01	Valid Gemini API configuration.	Application can send a recommendation request.
AI-02	API service temporarily unavailable.	Application shows a user-friendly failure message.
AI-03	API quota/rate limit is reached.	Application handles the failure without exposing sensitive configuration.
AI-04	AI response is returned with unexpected formatting.	Application handles or safely displays the response without crashing.
AI-05	Very short user request.	Application either generates a response using available information or requests more information.
AI-06	Detailed user request.	Important user constraints are passed to the AI request.

5. Boundary and Edge-Case Testing

- Extremely low budget values.
- Large budget values.
- Blank preferences.
- Long requirement descriptions.
- Special characters in user input.
- Unexpected spaces or formatting in input fields.
- Repeated requests.
- Temporary loss of internet connectivity.
- AI service returning an error or unavailable status.

6. UI and Usability Testing

- Verify that all labels and input controls are understandable.
- Verify that the generate/recommendation action is clearly visible.
- Verify that users receive feedback while an AI request is being processed.
- Verify that errors are readable and actionable.
- Verify that recommendation results are organized and easy to scan.
- Verify that the interface remains usable on different screen sizes.

7. Test Execution Record

Test ID	Result	Remarks
TC-01 to TC-08	To be recorded	Execute during final application testing.
AI-01 to AI-06	To be recorded	Execute with the configured Gemini API.
Edge Cases	To be recorded	Execute using boundary and failure scenarios.
UI Tests	To be recorded	Execute on supported desktop/mobile layouts.

8. Acceptance Criteria

PocketSmart AI is ready to proceed toward documentation and demonstration when the main user flow works end-to-end, valid inputs produce a usable recommendation response, invalid inputs are handled correctly, major AI/API failures are handled without application crashes, and the interface provides clear feedback to the user.

9. Testing Conclusion

The testing phase provides evidence that the implemented application behaves according to the planned requirements. Any defects identified during testing should be documented, corrected, and retested before the final demonstration.